Royal vs Frugal Amp



Zain Fathoni

Senior Software Engineer at BookThatApp.

SWE Growth Webinar
25 October 2026
13.00–14.30 WIB

https://zainfathoni.com

You do not need my budget to use Amp well.

Coding Agent Benchmark:
Using Amp with Royal vs Frugal mode

Not everyone has the same budget

Spending styles, not skill levels:

- **Sultan Token** — pays for speed and convenience
- **Mendang-mending** — chooses where to spend. That's me.
- **Kere Hore** — needs useful work on a small budget. Today is for you.

Royal

Frontier models + Amp-hosted Orbs. My Megawatt plan includes Orb quota.

Frugal

Strong planning where it matters. Cheaper workers on runners. No Orbs today.

Copy the setup. Inherit the bill.

The expensive part is not just the first answer.

Unclear tasks create retries, repair work, and more review.

A runner removes Orb usage, not model cost.

🧠 Models and tools

Planning, workers, review, and rescue. Credits, subscriptions, or provider bills.

🖥️ Compute and time

Hosted compute or your own machine. Plus the time you spend checking and fixing.

Small budget. Real confidence.

Imagine choosing one useful task, setting a budget, and knowing what a good result looks like.

You can start without buying my whole setup.

A task you can explain

One visible improvement. Clear limits. A result you can check.

A spend you choose

Set a cap before the run. Keep room for review and repair.

Cheap calls can still cost more

One prompt can delegate work. It does not guarantee clear boundaries.

More workers can mean more retries and more coordination.

Three worktrees can still share the **same port or test database.**

The tempting shortcut

Choose the cheapest model for every decision. Start more agents.

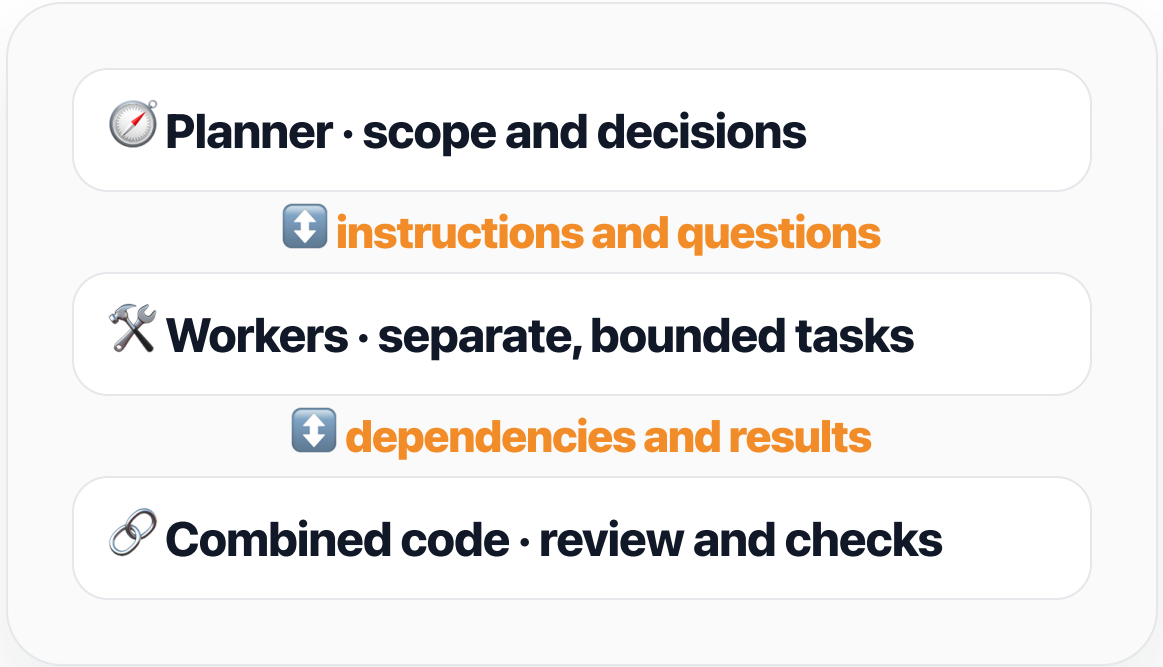
The missing work

Someone must define the tasks, resolve dependencies, and check the result.

Spend on judgment. Connect the workers.

Amp's standout feature: **native thread-to-thread communication.**

A stronger planner can guide cheaper workers, answer questions, and coordinate handoffs.



You already know the basics

Small tasks. Clear ownership. Shared contracts. Useful checks.

Use the same habits that help people work together.

🎯 Scope + contract

One useful result. Shared names and data.

🔒 Resource ownership

Assign files and runtime resources.

🧪 Checks

Know how to accept the result.

Choose one task. Set one cap.

1. Pick a small improvement you actually need.
2. Choose your spend cap and stop time.
3. Write the acceptance check.
4. Use a runner and a bounded worker.
5. Add parallel work only where tasks can separate.

> **Try this with your budget, not mine.**
>
> Our demo will show three tasks and their handoffs.

Your first win is control

- know what each worker owns
- get a question before a wrong assumption grows
- accept one useful change at a time

Less guesswork

You can see progress, decisions, and the next check.

A smaller next step

If a task stops, you can reduce scope, fix access, or choose help.

Make good work repeatable

> **You do not need my budget. You need to choose where your budget matters.**

Save through deliberate choices, not by making every step cheap.

Where to spend more

Scope, contracts, review of combined work, and rescue of blocked tasks.

Where to spend less

Bounded edits, repeated checks, and first drafts that a check can reject.

Today’s run order

TIME (WIB)	BLOCK	OUTPUT
13.00–13.15	Story, P1–P10	Shared frame
13.15–13.25	Budget, contract, launch	Ready workers
13.25–14.00	Workers and handoffs	Code for review
14.00–14.15	Integration, checks, results	What worked + cost
14.15–14.30	Q&A	Questions

Stop feature work at 14.00

Reduce scope if needed. Keep 15 minutes for integration and results.

LIVE DEMO · THREE USEFUL IMPROVEMENTS

One site. Three clear owners.

WORKER	USEFUL RESULT	ACCEPTANCE BOUNDARY
T1 · Blog	Find posts by category	Compose with `/blog`, `/blog/en`, `/blog/id`; preserve URLs and date order
T2 · Atlas	Browse curated topic groups	Link to blog categories; inspired by yegge.ai/atlas, not a graph or classifier
T3 · AMA	Ask a question on this site	Replace the dead homepage link; first-party form and private storage

Blog and Atlas share a category contract. AMA can proceed separately after its storage decision.

Choose the cap before the calls

- **Extra spend cap:** ____, chosen live
- **Review / rescue reserve:** ____, within the cap
- **Subscription allowance:** ____
- **Stop:** 14.00 WIB, or at the budget limit

Count the whole run

Planner + workers + review + rescue + paid tools.

Check each billing source

Amp credits, linked subscriptions, or provider API bills. Runner time is separate.

BEFORE LAUNCH · SHARED CONTRACT

Give Blog and Atlas the same starting point

AGREE FIRST	DELIVER BEFORE PARALLEL EDITS
Category names and slugs	One shared taxonomy and post schema
`categories: string[]`	Readable metadata, with defaults for existing posts
`?category=<slug>`	Same filter contract on all three blog routes
Empty / unknown category	Defined behavior and language-specific messages

T0 prepares and checks a foundation commit. Start T1 and T2 worktrees from it; each worker confirms the same revision.

BEFORE LAUNCH · AMA SCOPE

A private question form, not a platform

- Replace the homepage link and store questions privately.
- Server validation: required text and length limit.
- States: idle, sending, sent, failed.
- Basic abuse controls: hidden field, rate and size limits.
- No accounts, public feed, or admin dashboard.

The site stays on Vercel

Worker + D1 is a storage candidate, not a decision or a provisioned service.

Zain approves external resources

Test locally first. Private storage is not a promise of absolute anonymity.

LAUNCH · OPEN AMP NOW

Strong planner. Bounded workers.

- **Planner:** GPT-6.1 Sol in Medium, or Claude Code with Sonnet / Opus 5.5.
- **Workers:** Haiku 5.5 or GPT-6 Luna.
- **Runtime:** runner `zains`; one worktree per worker. No Orbs.

Read the actual selection

Mode, model, reasoning effort, and billing route. Confirm before the first message.

Launch after the handoff

Budget, foundation revision, ownership, and checks are ready.

Assign files and runtime resources

THREAD	OWNS	PORT PROPOSAL	DATA / BROWSER
T0 · Planner	Foundation, contract decisions, integration	5173	Combined checks
T1 · Blog	`posts.ts`, `BlogArchiveList.tsx`, blog routes	5174	Fixture posts; own session
T2 · Atlas	New Atlas route; reads shared post model	5175	Read-only posts; own session
T3 · AMA	`_index.tsx`, AMA route, API and private storage	5176	Own local state; own session

Confirm free ports and machine capacity. T1 owns post-model edits after the foundation handoff; T2 asks before contract changes.

A question can prevent three wrong edits

```
T0 → T1 + T2 · handoff
Start from the foundation commit I supplied. Confirm the revision and category schema
before editing.

T2 → T0 · dependency
Atlas needs an engineering group. Should it use an existing category or a new slug?

T0 → T1 + T2 · decision
Use the existing category. Keep the URL contract. T2 owns the display label.
```

Example exchange. Messages coordinate work; they do not automatically sync working copies.

Worktrees isolate source edits. Nothing else.

WORKTREES ISOLATE	WORKTREES DO NOT ISOLATE
source files	ports and processes
git branches	CPU and RAM
uncommitted edits	browser sessions
	shared databases and external services
	merge conflicts

Orbs isolate each thread's runtime. They still do not fix merge conflicts or shared external services.

Why I stayed with runners

I explored self-hosted Orbs in Palka.

The workspace-scoped setup did not fit my Personal workflow, so I stayed with runners.

Orbs remain useful when runtime isolation is worth the cost and setup.

Self-hosted Orbs

Provider registration is workspace-scoped. An individual can create a free workspace.

Compute still has a cost

You pay for your infrastructure. Today we use runners, with no Orbs.

One thread can reach several machines

Use **Rex** to reach another machine from the same Amp thread.

Create another thread for **a separate job**, not just machine access.

Keep thread communication for workers that own separate tasks.

```
# routes used in one thread
rex … # local Mac
rex -S vikas-rex …
rex -S zains-rex …
```

Three Macs, one Amp context. Avoid repeated thread setup; commands and output still use tokens.

INTEGRATION

Combine the code. Check the result.

1. Integrate ready branches from the same foundation.
2. Run typecheck and build together.
3. Test category × language, order, and empty states.
4. Follow every Atlas link.
5. Test AMA input, storage, errors, and abuse controls.

14.00 · Stop adding features

Accept only the work that passes combined checks. Keep unfinished work separate.

AFTER THE DEMO · ACTUAL RESULTS

What can we accept from this run?

TASK	COMBINED CHECK EVIDENCE	OUTCOME / REMAINING WORK
Blog categories	Fill from the live run	Pending
Curated Atlas	Fill from the live run	Pending
Private AMA	Fill from the live run	Pending

Show the result, one useful thread exchange, and the checks. Mark each task accepted, partial, or blocked.

Count planning, review, and rescue too

ROLE	MODEL / EFFORT	USAGE MEASURED	BILLING SOURCE	EXTRA CHARGE
T0 · Plan + review	Record live	Record live	Record live	Confirm / unknown
T1 · Blog	Record live	Record live	Record live	Confirm / unknown
T2 · Atlas	Record live	Record live	Record live	Confirm / unknown
T3 · AMA	Record live	Record live	Record live	Confirm / unknown
Rescue / paid tools	Record if used	Record if used	Record if used	Confirm / unknown

Also record existing plan fees, subscription limits, runner time, and human review time. Unknown charges are not zero.

TAKE THIS HOME

You do not need my budget

One useful task. A chosen limit. A clear check.

Pay for decisions. Use cheaper workers for clear tasks.

Let Amp threads ask, coordinate, and report.

> **Saving does not mean everything must be cheap. It means choosing what is worth paying for.**

References

- Amp — The Dial, Pricing, Agent to Agent
- Amp — Orbs and Self-Hosted Orbs
- SWE Growth — Coding Agent Benchmark agenda
- Dan Roam — The Pop-Up Pitch (this talk's storyline)
- Yegge Atlas — inspiration for task 2
- Zain Fathoni — You don't need CLI
- Zain's Rex multi-machine thread — skill + CLI example; thread access required

> **Choose the budget.**
>
> **Connect the work.**
>
> **Check the result.**

Q & A

zainfathoni.com/royal-vs-frugal-amp.html

ampcode.com

Zain Fathoni